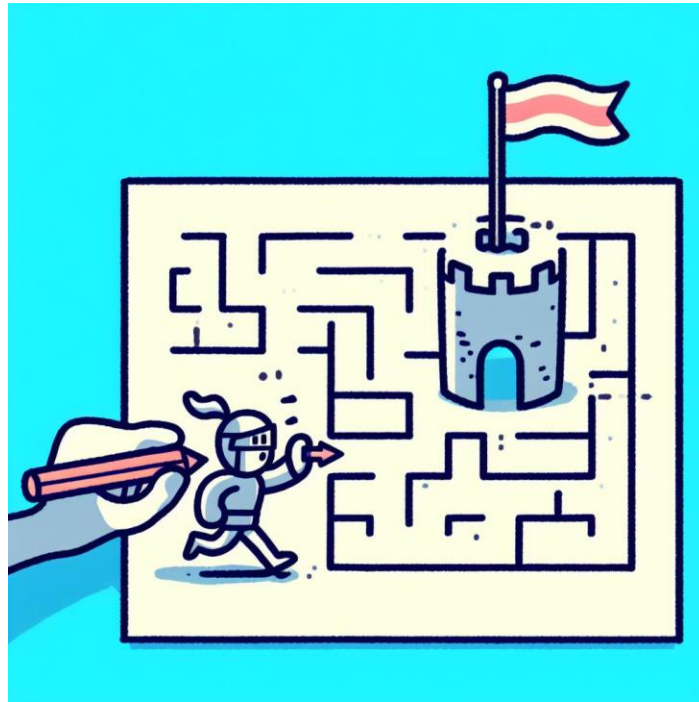


Práctica 1:

Búsqueda en el espacio de estados

Juan Alessandro Vázquez Bustos

Mario López García



Universidad Rey Juan Carlos
Desarrollo de juegos con inteligencia artificial

Índice

Índice	2
Introducción.....	3
Nodo	4
Algoritmo	5
Referencias	8

Introducción

La primera práctica de Desarrollo de juegos con inteligencia artificial nos proporciona un proyecto de Unity con un mapa compuesto de casillas y una serie de personajes y objetos. En el mapa hay diferentes tipos de casillas, debido a la existencia de casillas ocupadas por muros, que tienen enemigos, que conforman el límite del mapa, que contienen un tesoro o que son la salida del mapa.

El agente tiene la misión de llegar hasta la meta usando un algoritmo de búsqueda de estados. Para ello, se ha implementado un algoritmo usando una heurística débil, en contraposición a un algoritmo en amplitud o en profundidad, para conseguir un resultado óptimo con el menor coste, es decir, con la menor distancia.

El algoritmo tiene que calcular el camino a seguir mediante la expansión de nodos. Cuando avanza una casilla, el coste de ese movimiento es 1. Además, se hace uso de la heurística de la distancia euclídea. Ambos valores se suman para formar f^* , el valor que se usa para ordenar los nodos más prometedores en la lista abierta de nodos.

Una vez encuentra la meta, se realizará el camino inverso buscando cada padre del nodo meta gracias a contar con la referencia del nodo padre de cada nodo. Esa sucesión de nodos se guarda en un array de "CellInfo" que guiará el movimiento del agente.

A continuación, se procederá a explicar el funcionamiento del algoritmo implementado para que el agente encuentre la meta.

Nodo

Los algoritmos de búsqueda en el espacio de estados utilizan nodos. Es por ello que, para la correcta implementación de nuestro algoritmo, es esencial la creación de una clase 'Nodo'.

Cada nodo dispone de las siguientes propiedades: 'infoCelda' (información de la celda en la que está el nodo), 'informacionMundo' (un objeto de clase 'WorldInfo'), 'padre' (una referencia al nodo sucesor), 'g' (el coste de llegar al nodo desde el nodo inicial) y 'fEstrella' (el valor de la heurística, la distancia desde el nodo hasta la meta usando la distancia euclídea).

```
private CellInfo infoCelda;  
private WorldInfo informacionMundo;  
private Nodo padre;  
private float g;  
private float fEstrella;
```

Para acceder a todas estas propiedades desde el exterior, hemos implementado getters.

La variable 'fEstrella' obtiene su valor en función del nº de iteraciones que han llevado a un nodo (g) y de la distancia euclídea a la meta desde dicha casilla. Este valor F* se calcula directamente en el constructor de la clase, en el momento de crear el nodo:

```
fEstrella = g +  
this.infoCelda.Distance(this.informacionMundo.Exit,  
CellInfo.DistanceType.Euclidean);
```

Una de las funciones esenciales de la clase 'Nodo' es poder expandir dicho nodo a sus adyacentes. Para ello, hemos creado un método 'expandirNodo' que devuelve una lista de nodos con los nodos vecinos del nodo que la utiliza. Para conseguir acceder a estos nodos vecinos es necesario disponer de la información del mundo: A continuación, se muestra a modo de ejemplo como se crea uno de los nodos expandidos.

```
CellInfo derecha = informacionMundo[infoCelda.x+1, infoCelda.y];
```

Al crear el nuevo nodo vecino, es importante tener en cuenta el incremento del coste, ya que el agente habrá caminado una casilla más, de ahí que g se vea incrementado:

```
Nodo nodoDerecha = new Nodo(informacionMundo, derecha, this, g+1);
```

Por último, antes de añadir dicho nodo a la lista que más adelante se devolverá, es necesario comprobar que por las celdas que contienen es posible caminar, es decir, que son 'Walkable':

```
if (derecha.Walkable)  
{  
    nodosExpandidos.Add(nodoDerecha);  
}
```

La clase 'Nodo' también dispone de otro método público, llamado 'esMeta', que comprueba si el nodo está en la meta. Lo hace comparando sus respectivos objetos 'CellInfo', determinando si son iguales:

```
CellInfo meta = informacionMundo.Exit;  
return meta.Equals(this.infoCelda);
```

Algoritmo

Para el algoritmo, se ha creado una nueva clase llamada 'aEstrellaMovement' que implementa la interfaz 'INavigationAlgorithm'. Esta interfaz obliga a que nuestra clase contenga dos métodos importantes: 'Initialize' y 'GetPath'.

Las variables globales de la clase son las siguientes: '_mundo' (la información del mundo), '_listaAbierta' (la lista abierta del algoritmo de búsqueda), '_listaCerrada' (la lista de nodos visitados para evitar todos los estados repetidos), '_padresMeta' (la lista donde se guardan los nodos antecesores a la meta para crear el trayecto) y '_meta' (un booleano que indica si se ha llegado a la meta o no). La variable 'factor' modifica el tamaño máximo de la lista que se tiene en cuenta a la hora de ordenarla.

```
private WorldInfo _mundo;  
private List<Nodo> _listaAbierta = new List<Nodo>();  
private List<Nodo> _listaCerrada = new List<Nodo>();  
private List<Nodo> _padresMeta = new List<Nodo>();  
private bool _meta = false;  
private int factor = 5;
```

El método 'GetPath' nos da dos parámetros fundamentales para el cálculo de la ruta: el nodo inicial 'startNode' y el nodo meta 'targetNode'. Para el correcto funcionamiento, debemos devolver un array de 'CellInfo'. Partiendo de esta información se puede proceder a la elaboración del algoritmo. Para realizar el algoritmo nos basamos en el código proporcionado por el profesor en el Aula Virtual.

Primero, se crean instancias de la clase 'Nodo' para el nodo inicial y el nodo final.

```
Nodo nodoInicial = new Nodo(_mundo, startNode, null, 0);  
Nodo nodoFinal = new Nodo(_mundo, targetNode, null, 0);
```

Al inicio, la lista abierta estará vacía. Debemos incluir el primer nodo, el nodo inicial, antes del comienzo del bucle:

```
_listaAbierta.Add(nodoInicial);
```

Para hacer la expansión de nodos hasta encontrar la meta usamos un bucle while. Lo primero que hacemos es guardar la referencia al primer elemento de la lista abierta y lo eliminamos de dicha lista:

```
actual = _listaAbierta[0];  
_listaAbierta.RemoveAt(0);
```

Para evitar todos los estados repetidos, necesitamos llevar el control de los nodos visitados. Insertamos el nodo visitado a la lista cerrada:

```
_listaCerrada.Add(actual);
```

Antes de expandir el nodo a sus adyacentes, debemos comprobar si es un nodo meta. En el caso de que lo sea, el bucle while debe detenerse y se procederá a calcular la ruta. Se usa la función 'Equals' para comprobar si el nodo actual está en la misma casilla que el nodo objetivo, el nodo final:

```
if (actual.getInfoCelda().Equals(nodoFinal.getInfoCelda()))  
{  
    _padresMeta.Add(actual);  
    while (actual.getPadre() != null)  
    {  
        _padresMeta.Add(actual.getPadre());  
        actual = actual.getPadre();  
    }  
    _meta = true;  
}
```

Al cambiar el valor de la variable booleana '_meta' a true, le comunicamos al algoritmo que ya ha encontrado y la meta. El bucle while no continuará. Este bloque if tiene otra función importante: crea la ruta que seguirá el agente, aunque a la inversa. Añade la meta, después añade el nodo padre de la meta, y así sucesivamente hasta llegar al nodo inicial. Lo único que resta hacer para tener la ruta es invertir el orden de esta sucesión de nodos, pero eso es algo que se realizará más adelante.

A continuación, se explica el caso más común, el caso en el que el nodo no se sitúa en la meta. En este caso el nodo actual debe ser expandido. Es fundamental comprobar aquí la existencia de estados repetidos. Creamos la lista de nodos expandidos:

```
List<Nodo> nodosExpandidos = actual.expandirNodo();
```

Aquí se muestra la forma de descartar los nodos que ya se encuentran en la lista cerrada:

```
for (int i = 0; i < _listaCerrada.Count; i++)  
{  
    if (nodo.getInfoCelda().Equals(_listaCerrada[i].getInfoCelda()))  
    {  
        count = true;  
        break;  
    }  
}
```

Una vez se ha decidido qué nodos pasan el filtro de la lista cerrada y cuáles no en el caso de que la lista no contenga nodos, este simplemente se añadirá a esta. Para el caso en el que sí que tenga

nodos se ha implementado un for que recorre los elementos de la lista abierta mientras compara si el valor f^* del nodo a añadir a la lista es menor al de alguno de los nodos que ya existe en la lista abierta para en ese caso insertarlo delante suyo.

Se compara un máximo de 'factor' elementos en la lista abierta. Se puede modificar esta variable 'factor' en las declaraciones iniciales. A mayor sea el valor de esta, más nodos de la lista abierta se compararán de forma que más elementos quedarán insertados en orden de coste.

```
for (int i = 0; i < Math.Min(_listaAbierta.Count, factor); i++) {  
    if (nodo.getFEstrella() < _listaAbierta[i].getFEstrella())  
    {  
        _listaAbierta.Insert(i, nodo);  
        entre2 = true;  
        break;  
    }  
}
```

En el caso de que se sobrepase el límite factor o el número de elementos de la lista abierta y el f^* del nodo no sea menor a ninguno con los que se ha comparado, dicho nodo será insertado al final de la lista abierta.

En la parte final del código, una vez el while ya ha finalizado, se prepara la ruta para devolverla en forma de array de 'CellInfo'. Primero se crea un array del mismo tamaño que la lista de padres '_padresMeta':

```
CellInfo[] path = new CellInfo[_padresMeta.Count];
```

El siguiente bloque for tiene el objetivo de invertir la lista a la vez que introduce la propiedad de 'CellInfo' de los elementos en el array 'path'. Para ello, el bucle recorre desde el índice más alto de la lista hasta el índice 0 y los va recorriendo haciendo uso de dos variables "i" y "j":

```
for (int i = _padresMeta.Count-1, j = 0; i >= 0; i--, j++)  
{  
    Debug.Log("Iteracion nº "+j+ " f* = " + _padresMeta[i].getFEstrella());  
    path[j] = _padresMeta[i].getInfoCelda();  
}
```

Por último, devolvemos el array con la ruta:

```
return path;
```

Referencias

Transparencias del tema 3. (s. f.). [Diapositivas]. Aula Virtual.

Creación de la portada de la memoria: [Generador de imágenes de Bing.](#)